

Da auto-avaliação à instrumentação automatizada

No Sprint 7 vocês fizeram assessment de maturidade contra TMMi e DORA 2024, e o exercício expôs lacunas concretas e mensuráveis nas áreas de processo do TMMi Level 2 — especialmente Test Policy and Strategy, Test Design and Execution documentadas e, sobretudo, ausência de rastreabilidade formal entre requisito, caso de teste e evidência. O Sprint 8 ataca essas lacunas exatas, mas não como exercício documental no Word — como instrumentação automatizada no repositório de código. A escalada é deliberada: deixa-se de descrever maturidade e passa-se a implementá-la com ferramentas de mercado.

Quatro camadas instrumentais articuladas

Camada 1 — Especificação formal pelo padrão internacional vigente: ISO/IEC/IEEE 29119-3:2021, que sucedeu o IEEE 829-2008 como referência mundial para documentação de teste de software (ISO, 2021).

Camada 2 — Testes de contrato com Schemathesis sobre OpenAPI 3.1: a evolução property-based do Sprint 3 agora aplicada ao nível HTTP, gerando milhares de requisições automaticamente a partir do schema da API (DYGALO, 2024).

Camada 3 — Matriz de rastreabilidade requisito ↔ caso ↔ evidência automatizada via GitHub Actions: o CI deixa de ser apenas executor de testes e passa a ser orquestrador de evidência de auditoria, com o pipeline gerando a matriz a cada commit.

Camada 4 — Position paper técnico em diálogo com literatura contemporânea sobre continuous testing maturity, com leitura obrigatória de Nayak et al. (2024) publicado pelo IEEE em 2024 sobre sustentabilidade de pipelines de teste contínuo.

Conceito essencial: a virada deste sprint

Nos Sprints 1 a 3 a pergunta foi: o sistema responde?

No Sprint 4: minha suíte detectaria?

No Sprint 5: meus mutantes são matáveis?

No Sprint 6: como se chama o que produzi?

No Sprint 7: em que estágio de maturidade me posiciono perante o setor financeiro internacional?

Neste Sprint 8 a pergunta muda definitivamente: a evidência de que o meu processo de teste opera com rigor industrial é produzida automaticamente pelo CI a cada commit? A virada é da declaração para a demonstração — o pipeline é a prova viva.

Roteiro Sprint 8 — especificação ISO/IEC/IEEE 29119-3, contratos com Schemathesis e matriz de rastreabilidade automatizada via GitHub Actions ou similar

Como ler este roteiro

Este roteiro tem uma lógica de progressão. Cada bloco prepara o seguinte. Você vai encontrar blocos de cinco naturezas, e precisa tratá-los de formas diferentes:

Blocos azuis trazem conceito ou continuidade. Leia com atenção; não há resposta a registrar.

Blocos rosa trazem investigação técnica do aluno. Você precisa registrar sua resposta no arquivo Word de entrega antes de avançar. A resposta é avaliada por consistência com os autores e padrões indicados (ISO, 2021; DYGALO, 2024; NAYAK et al., 2024; OpenAPI Initiative; GitHub Docs) e pela qualidade do raciocínio.

Blocos amarelos trazem atenção técnica ou troubleshooting.

Blocos roxos trazem comandos, prompts ou esqueletos de configuração para adaptar. NUNCA cole o conteúdo bruto — você precisa preencher com os dados do seu repositório.

Blocos verdes trazem resultado esperado ou reflexão final. Eles são especialmente importantes — é como você sabe que está no caminho certo sem o professor ao lado.

Atenção: este sprint exige conta gratuita no GitHub ou similar. As fontes primárias estão em acesso aberto (URLs nas Referências). Respostas sem ancoragem nos padrões e autores indicados, sem evidência empírica de pipeline rodando no seu repositório, ou construídas como paráfrase superficial dos textos lidos não serão consideradas tecnicamente válidas.

Objetivo deste sprint

Você vai produzir, ao final deste sprint, três artefatos profissionais articulados que qualquer testador júnior contratado em fintech ou empresa de software regulada precisa saber entregar: (I) caderno de especificação de casos de teste em conformidade com o padrão

internacional ISO/IEC/IEEE 29119-3:2021; (II) suíte de contract testing rodando com Schemathesis contra a API do internet banking via schema OpenAPI 3.1; (III) pipeline CI no GitHub Actions que executa toda a sua suíte agregada dos Sprints 1 a 5 mais o Schemathesis do Sprint 8 e gera automaticamente, a cada commit, a matriz de rastreabilidade requisito ↔ caso de teste ↔ evidência de execução. A entrega final tem ainda um position paper técnico de duas páginas em diálogo crítico com Nayak et al. (2024).

Antes de começar — pré-requisitos obrigatórios

- ☐ Sprints 1 a 7 concluídos com arquivos Word de entrega salvos.
- ☐ Pasta `internet_banking_tests` intacta com a suíte agregada do Sprint 5.
- ☐ Conta gratuita no GitHub (github.com) e Git instalado e configurado na máquina.
- ☐ Acesso ao AVA (ambiente.virtual.ifro.edu.br) com o Roteiro Sprint 8 e os PDFs das leituras obrigatórias.
- ☐ Capacidade de leitura técnica em inglês para o padrão ISO/IEC/IEEE 29119-3 (resumo executivo em acesso aberto) e para Nayak et al. (2024).
- ☐ Software para edição de YAML (VSCode já basta) e Python 3.11+ no ambiente `venv` ativado.

Camada 1 — Conceito: ISO/IEC/IEEE 29119-3:2021 — Test Documentation

A ISO/IEC/IEEE 29119-3:2021 é a norma internacional vigente para documentação de teste de software. Sucedeu formalmente o IEEE Standard 829 (1998 e 2008) e foi publicada conjuntamente pela ISO, pela IEC e pelo IEEE — três das maiores organizações de padronização do mundo (ISO, 2021). Disponível para consulta em <https://www.iso.org/standard/81293.html>.

A norma estabelece templates para os seguintes documentos de teste, articulados em três níveis (organizational, project, dynamic):

Test Policy: intenção estratégica da organização quanto à qualidade dos testes.

Test Plan: escopo, abordagem, recursos e cronograma da campanha de teste do projeto.

Test Design Specification: técnicas, condições de teste e cobertura projetada.

Test Case Specification: identificador, descrição, pré-condições, dados de entrada, passos, resultados esperados, condições de saída, dependências, técnica e rastreabilidade ao requisito.

Test Procedure Specification: passos operacionais para executar um conjunto de casos de teste.

Test Log e Test Incident Report: evidência de execução e registro de incidentes detectados.

Esta é a linguagem documental que auditorias, certificações TMMi e regulações setoriais (BACEN, ANS, ANATEL) esperam encontrar quando avaliam um processo de teste. Não saber especificar nesse padrão é estar fora da conversa profissional.

Investigação Técnica 1 — produção do caderno de Test Case Specification

Você vai especificar formalmente quatro casos de teste da sua suíte do internet banking no formato ISO/IEC/IEEE 29119-3 simplificado, integrando o vocabulário BSTQB consolidado no Sprint 6 (níveis e técnicas) e o posicionamento TMMi do Sprint 7 (área de processo de referência) nos campos de classificação e rastreabilidade — a Investigação Técnica 1 do Sprint 8 é, portanto, a primeira em que os sete sprints anteriores aparecem articulados em um único artefato. Selecione um caso por cada um dos seguintes artefatos: `test_ia_gerado.py` (Sprint 1), `test_hypothesis.py` (Sprint 3), `test_melhorado.py` (Sprint 4), `features/transferencia.feature` (Sprint 5).

Para cada caso, preencha o template mínimo a seguir, em uma tabela no seu arquivo Word de entrega:

ID do caso de teste:

Descrição em uma frase:

Pré-condições (estado inicial necessário):

Dados de entrada (valores concretos):

Passos de execução (numerados):

Resultado esperado (status HTTP, mensagem, estado final do banco):

Condições de saída (limpeza necessária):

Dependências de outros casos:

Nível BSTQB (do Sprint 6 — unidade, integração, sistema ou aceitação):

Técnica BSTQB aplicada (do Sprint 6 — caixa preta, caixa branca ou baseada em experiência, com subtécnica):

Área de processo TMMi de referência (do Sprint 7 — Test Policy and Strategy, Test Planning, Test Monitoring and Control, Test Design and Execution ou Test Environment):

Requisito do internet banking ao qual rastreia:

Critério de avaliação: os campos precisam ser preenchidos com precisão técnica observável no código real do seu artefato — não inventados. Um avaliador externo deve conseguir reproduzir a execução manualmente a partir da sua especificação.

Camada 2 — Conceito: Schemathesis e contract testing sobre OpenAPI

Schemathesis é a ferramenta open-source que aplica testes baseados em propriedade automaticamente a APIs HTTP a partir do seu schema OpenAPI (DYGALO, 2024). Documentação oficial em <https://schemathesis.readthedocs.io/>.

É a evolução direta do que vocês fizeram com Hypothesis no Sprint 3: lá, vocês formularam invariantes manualmente e o Hypothesis gerava valores. Aqui, vocês descrevem a API uma única vez em OpenAPI 3.1 (especificação em <https://spec.openapis.org/oas/v3.1.0>) e o Schemathesis gera milhares de requisições HTTP automaticamente, testando se a API respeita o próprio contrato declarado — uma forma rigorosa de teste de aceitação contratual no sentido do BSTQB (2023, Cap. 2).

Para este sprint, vocês escreverão o `openapi.yaml` manualmente, observando o comportamento atual da API e traduzindo-o em contrato declarativo formal. Esse exercício é o coração da Camada 2 — especificação de contrato observado é, ela mesma, um artefato de teste, contraparte HTTP do que a Camada 1 faz com casos de teste pelo padrão 29119-3. O ato de escrever o YAML obriga a declarar explicitamente cada endpoint, cada parâmetro, cada código de resposta e cada formato de payload; essa declaração explícita é o oráculo que o Schemathesis usará para gerar centenas de requisições e detectar divergências entre contrato e comportamento real.

Atividade prática — instalação e execução do Schemathesis

Com o `venv` ativado dentro de `internet_banking_tests`, execute no terminal:

```
pip install schemathesis
```

Crie um arquivo `openapi.yaml` na raiz da pasta com a descrição mínima dos três endpoints — `GET /saldo/{conta_id}`, `POST /transferencia`, `GET /extrato/{conta_id}` — com tipos, parâmetros, request body e respostas esperadas (HTTP 200, 404, 422). Use a especificação OpenAPI 3.1 como referência.

Inicie o servidor da API em um terminal:

```
python app.py
```

Em outro terminal, rode o Schemathesis contra a API local apontando para o `openapi.yaml`:

```
schemathesis run openapi.yaml --base-url=http://127.0.0.1:5000 --checks all --hypothesis-max-examples=100
```

Schemathesis vai gerar centenas de requisições HTTP com valores variados a partir do schema, e reportar qualquer divergência entre o contrato declarado e o comportamento real da API.

Investigação Técnica 2 — análise dos achados do Schemathesis

Registre no seu arquivo Word de entrega:

- O conteúdo completo do seu `openapi.yaml` (copie e cole).
- Output completo do `schemathesis run` (copie do terminal).
- Para cada divergência reportada pelo Schemathesis entre contrato e comportamento, classifique a causa segundo a taxonomia do Sprint 2 (A erro de teste / B erro de API / C defeito real / D erro de isolamento) e justifique.
- Em uma frase: qual é a diferença epistemológica entre o que o Hypothesis do Sprint 3 verificou (invariantes formuladas por humano) e o que o Schemathesis verifica neste sprint (contrato declarativo da API)?

Camada 3 — Conceito: matriz de rastreabilidade automatizada no CI

Matriz de rastreabilidade requisito ↔ caso de teste ↔ evidência é um instrumento clássico de auditoria, exigido por TMMi Level 3 e por normas regulatórias bancárias. Historicamente, era mantida em planilhas Excel atualizadas manualmente — fonte permanente de inconsistência entre o que estava no documento e o que estava no código.

A escalada moderna é gerar a matriz automaticamente no pipeline CI, a cada commit, lendo (i) os IDs de teste especificados no caderno 29119-3, (ii) os resultados de execução do

pytest agregado e (iii) os requisitos rastreados via marcadores (pytest markers ou docstrings padronizadas). GitHub Actions é a plataforma CI mais usada do mundo open-source em 2024-2025 (documentação oficial em <https://docs.github.com/en/actions>).

A virada conceitual é importante: o CI deixa de ser apenas executor da suíte e passa a ser produtor de evidência de auditoria. A maturidade do processo é demonstrada a cada commit, não declarada em reunião.

Atividade prática — esqueleto do pipeline GitHub Actions

Crie um repositório novo no GitHub chamado `internet_banking_tests` (público ou privado), faça commit do conteúdo da sua pasta local. Dentro da pasta, crie o subdiretório `.github/workflows/` e nele um arquivo `test.yml` com o seguinte esqueleto (adapte ao seu Python, ao seu `requirements.txt` e ao seu `openapi.yaml`):

```
name: Testes do internet banking
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.11' }
      - run: pip install -r requirements.txt
      - run: pip install schemathesis
      - run: python app.py &
      - run: sleep 3
      - run: pytest -v --junit-xml=report.xml
      - run: schemathesis run openapi.yaml --base-url=http://127.0.0.1:5000 --checks
all
      - run: python scripts/gera_matriz.py > matriz_rastreabilidade.md
      - uses: actions/upload-artifact@v4
        with: { name: evidencia, path: 'report.xml matriz_rastreabilidade.md' }
```


O script `gera_matriz.py` é responsabilidade sua — escreva um Python simples que lê o `report.xml` do `pytest` e os IDs de teste do seu caderno 29119-3 e produz a tabela em Markdown.

Investigação Técnica 3 — execução do pipeline e análise do resultado

Após commit e push do `test.yml` e do `gera_matriz.py` para o GitHub, abra a aba Actions do seu repositório e acompanhe a execução. Registre no seu arquivo Word:

- URL pública do seu repositório no GitHub.
- Print da aba Actions mostrando o último run com status verde (ou vermelho, se houver falha — não esconda).
- Conteúdo da `matriz_rastreabilidade.md` gerada como artefato pelo pipeline (copie e cole).
- Em uma frase: o que mudou na sua relação com a expressão "qualidade demonstrável" depois de ver o pipeline gerar a matriz automaticamente?

Atenção — armadilhas comuns na configuração do CI

Primeiro: o servidor da API precisa rodar em background no runner (uso de "&" no comando) e dar tempo de inicialização (`sleep 3`) antes de Schemathesis tentar conectar — sem isso, o pipeline falha por `connection refused`. Segundo: o `requirements.txt` precisa listar todas as dependências do app e dos testes (incluindo as instaladas ao longo dos Sprints 1 a 5 — `pytest`, `hypothesis`, `requests`, `pytest-bdd` — mais `schemathesis` deste sprint) — se faltar uma, o runner falha em `pip install`. Terceiro: `actions/upload-artifact` espera caminhos separados por espaço em string única, não como lista YAML — formate como mostrado. Quarto: se o repositório for privado, garanta que a aba Actions está habilitada nas Settings.

Camada 4 — Conceito: literatura sobre continuous testing maturity

Esta camada exige que vocês operem como leitores críticos de literatura primária — mesmo movimento do Sprint 7, agora aplicado ao domínio de continuous testing.

Leitura obrigatória: NAYAK, K. et al. (2024) — Sustainable Continuous Testing in DevOps Pipeline, publicado em IEEE Xplore em 2024. O paper analisa o consumo energético

e a sustentabilidade de execuções não atendidas de testes em pipelines CI/CD e propõe critérios para limitar runs desnecessários sem comprometer a capacidade detectora.

O paper é relevante para a sua experiência porque vocês acabaram de implementar exatamente o tipo de pipeline que ele analisa — e a leitura crítica lhes dá vocabulário para discutir trade-offs reais que aparecem quando o pipeline roda dezenas ou centenas de vezes por dia em produção.

Investigação Técnica 4 — relatório técnico de duas páginas (entrega central)

Produza um relatório técnico de duas páginas em formato acadêmico (Times New Roman 12, espaçamento 1,5, justificado, citações em ABNT), com a estrutura mínima a seguir:

Título: tese específica, defensável. Ex.: "A automação da matriz de rastreabilidade no CI desloca, mas não elimina, a necessidade de auditoria humana — três evidências do meu pipeline."

Parágrafo 1 — Contexto e tese (até 8 linhas): qual é a sua tese central, em uma frase.

Parágrafo 2 — Evidência do seu pipeline (até 15 linhas): narre dois ou três episódios concretos da configuração e execução do seu CI que sustentam a tese. Cite logs, arquivos e tempos reais.

Parágrafo 3 — Diálogo com Nayak et al. (2024) (até 15 linhas): concorda com os autores? Em que ponto? Que aspecto da sua experiência empírica eles não capturam?

Parágrafo 4 — Proposta (até 12 linhas): que prática concreta de configuração de CI você proporia para fechar a lacuna identificada? Seja específico — não "melhorar o CI".

Parágrafo 5 — Limites do argumento (até 8 linhas): onde sua tese é frágil? Que evidência adicional seria necessária para sustentá-la em produção real?

Referências: no mínimo cinco entradas em ABNT, das quais pelo menos três publicadas em 2023-2025.

Síntese epistemológica — registre em seu arquivo Word

Responda em três frases curtas:

- a) Em que mudou, operacionalmente, a sua relação com a expressão "evidência de qualidade" entre o Sprint 1 (output do pytest no terminal) e o Sprint 8 (matriz gerada automaticamente pelo CI)?
- b) Qual achado do Schemathesis mais surpreendeu — algo que sua suíte manual dos Sprints 1 a 5 não havia detectado?
- c) Em uma frase, qual é a sua tese para o relatório técnico antes mesmo de escrevê-lo?

Entrega no AVA

Arquivo Word ou PDF único contendo, na ordem:

- 1) Caderno de Test Case Specification (Investigação Técnica 1) com quatro casos especificados pelo padrão ISO/IEC/IEEE 29119-3 simplificado.
- 2) Conteúdo do openapi.yaml e output do schemathesis run com análise dos achados (Investigação Técnica 2).
- 3) URL pública do repositório GitHub, print da aba Actions com run verde e conteúdo da matriz_rastreabilidade.md (Investigação Técnica 3).
- 4) Relatório técnico de duas páginas em diálogo crítico com Nayak et al. (2024) (Investigação Técnica 4).
- 5) Síntese epistemológica em três frases.

Pontuação: até 10 pontos. Distribuição: 2 pontos pelo caderno 29119-3 com precisão técnica; 2 pontos pelo openapi.yaml válido e análise dos achados do Schemathesis; 3 pontos pelo pipeline rodando verde no GitHub com matriz gerada automaticamente; 2 pontos pelo relatório técnico; 1 ponto pela síntese epistemológica.

Pipeline não funcional no GitHub, openapi.yaml inválido, casos de teste especificados sem ancoragem no código real ou relatório sem tese própria não serão considerados tecnicamente válidos.

Referências Bibliográficas

DYGALO, D. *Schemathesis: Property-based testing for OpenAPI and GraphQL APIs*. Documentação oficial, 2024. Disponível em: <https://schemathesis.readthedocs.io/>. Acesso em: junho 2026.

GITHUB. *GitHub Actions Documentation*. GitHub, 2024. Disponível em: <https://docs.github.com/en/actions>. Acesso em: junho 2026.

ISO — INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. *ISO/IEC/IEEE 29119-3:2021 — Software and systems engineering — Software testing — Part 3: Test documentation*. Geneva: ISO, 2021. Disponível em: <https://www.iso.org/standard/81293.html>. Acesso em: junho 2026.

NAYAK, K.; ROUTE, S.; SUNDARARAJAN, M.; JAIN, A.; R., S. Sustainable Continuous Testing in DevOps Pipeline. In: IEEE Conference Publication, 2024. Disponível em: https://www.researchgate.net/publication/382512542_Sustainable_Continuous_Testing_in_DevOps_Pipeline. Acesso em: junho 2026.

OPENAPI INITIATIVE. *OpenAPI Specification v3.1.0*. OpenAPI Initiative / Linux Foundation, 2021. Disponível em: <https://spec.openapis.org/oas/v3.1.0>. Acesso em: junho 2026.

Referências legadas dos Sprints 1 a 7 — mantidas como contexto

BSTQB — BRAZILIAN SOFTWARE TESTING QUALIFICATIONS BOARD. *Certified Tester Foundation Level Syllabus*: versão 4.0. [S. l.]: ISTQB; BSTQB, 2023. Disponível em: https://www.bstqb.online/files/syllabus_ctfl_4.0br.pdf. Acesso em: maio 2026.

DORA / GOOGLE CLOUD. *Accelerate State of DevOps Report 2024*. Google Cloud, 2024. Disponível em: <https://dora.dev/research/2024/dora-report/>. Acesso em: junho 2026.

TMMi FOUNDATION. *TMMi Framework*: Release 1.2. TMMi Foundation, 2018. Disponível em: <https://www.tmmi.org/tmmi-framework/>. Acesso em: junho 2026.